```
$ export LANGSMITH_TRACING=true
```

# LangSmith for Developers

Configure, observe, debug and evaluate your AI apps — from your first trace to production monitoring

**Basics → Advanced** | 8-step hands-on lab included

Abinash Kumar Mishra · Developer Training · October 2026

THE LEARNING PATH

# Seven stops, from first trace to production

01

### Basics

Why observability, traces, runs, run types

02

### Configure

API key, env vars, three ways to instrument

03

### Observe

UI tour, filters, cost, dashboards, feedback

04

### Debug

Symptom-to-span playbook, Playground

05

### Evaluate

Datasets, evaluators, experiments

06

### Production

Masking, sampling, environments, the FIL route

07 · HANDS-ON LAB

### Help Desk Assistant

8 steps, ~90 minutes. Works with an OpenAI key or a free built-in mock LLM.

01 · BASICS

# Why AI apps need their own observability

### Silent failures

A wrong answer returns HTTP 200. No stack trace, no alert — just an unhappy user.

### Non-deterministic

Same input, different output. You can't reproduce a bug without the exact prompt and context.

### Many hidden steps

Retriever → prompt → LLM → tool → LLM. The bug can hide in any hop of a RAG or agent flow.

### Cost and latency

Every token is billed. One runaway agent loop can burn a day's budget in minutes.

**Logs tell you that something happened. Traces tell you why.**

01 · BASICS

# What LangSmith is

A platform to trace, monitor, evaluate and improve LLM applications — built by the LangChain team, but it works with any framework or none.

### Observability

Traces, threads, dashboards, alerts, cost tracking

### Evaluation

Datasets, evaluators, experiments, online evals, annotation queues

### Prompt engineering

Playground, versioned prompts, side-by-side model comparison

### Deployment

Cloud, hybrid or self-hosted; agent deployment

SDKs: `Python` `TypeScript` `Java` `Kotlin` `OpenTelemetry`

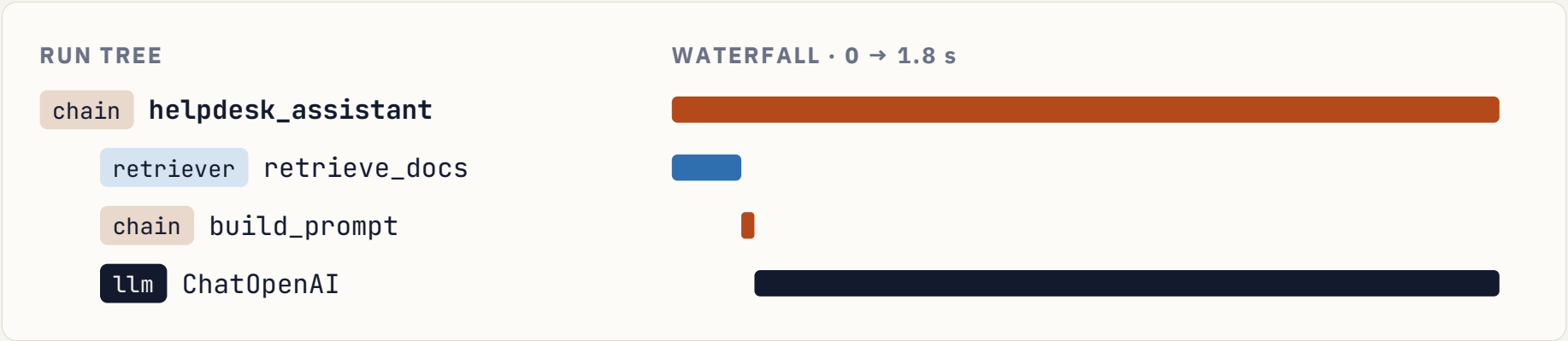
01 · BASICS

# Seven words you'll use every day

| Term | What it is | Analogy |
|---|---|---|
| Project | A bucket of traces — one per app and environment | A log index |
| Trace | Everything that happened for one request | A distributed trace |
| Run | One step inside a trace: an LLM call, a tool, a retriever | An OpenTelemetry span |
| Thread | Traces linked into one conversation by a session id | A chat session |
| Feedback | A score attached to a run — from a user, a human or code | A star rating |
| Dataset | Saved inputs plus expected outputs used for testing | A test fixture |
| Experiment | One run of your app over a dataset, scored by evaluators | A test-suite run |

01 · BASICS

# Anatomy of a trace: a tree of runs over time




**Trace** = the whole tree for one request, sharing one trace id.

**Each row is a run** with its own inputs, outputs, latency, error, tokens.

**Read the waterfall**: the longest bar is where the time goes.

01 · BASICS

# Run types tell LangSmith how to show a step

| run_type | Use it for | UI bonus |
|---|---|---|
| chain | Default: any function or pipeline step | Inputs / outputs |
| llm | A model call | Messages, tokens, cost |
| retriever | Vector or keyword search | Document list view |
| tool | Functions an agent calls | Tool-call view |
| prompt | Prompt formatting | Rendered prompt |
| parser | Output parsing | Inputs / outputs |

**RULE OF THUMB**

Set **run_type** correctly and LangSmith does the rendering for you.

An **llm** run without token usage shows no cost. A **retriever** should return a list of documents with page_content and metadata.

02 · CONFIGURE

# Get connected in three steps

### 1 · Account

Sign up at **smith.langchain.com**. The free Developer plan is enough for this course.

### 2 · API key

Settings → API Keys → Create. Keys start with **lsv2_**. Treat it like a password.

### 3 · Env vars

Put them in **.env**, load before the app starts, and never commit the file.

```
# install the SDK
pip install -U langsmith openai python-dotenv

# .env
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_pt_xxxxxxxx
LANGSMITH_PROJECT=helpdesk-dev
```

02 · CONFIGURE

# Environment variables: the control panel

| Variable | Purpose | Needed? |
|---|---|---|
| LANGSMITH_TRACING | Master on/off switch (true) | Yes |
| LANGSMITH_API_KEY | Authenticates the SDK | Yes |
| LANGSMITH_PROJECT | Which project receives traces | Strongly advised |
| LANGSMITH_ENDPOINT | EU region or self-hosted URL | Sometimes |
| LANGSMITH_WORKSPACE_ID | Pick a workspace when a key spans many | Sometimes |
| LANGSMITH_TRACING_SAMPLING_RATE | Keep a fraction of traces, e.g. 0.1 | Production |
| LANGSMITH_HIDE_INPUTS / _OUTPUTS | Don’t send payloads at all | Sensitive data |

No **LANGSMITH_PROJECT**? Traces land in a project called **default** — fine for a demo, chaos for a team.

02 · CONFIGURE

# Three ways to instrument an app

A · FRAMEWORK

## LangChain / LangGraph

Env vars only. Every chain, node, tool and model call is traced.

```
# no code change
graph.invoke(state)
```

B · WRAPPER

## Wrap the SDK client

One line. Captures messages, model, tokens and cost.

```
from langsmith.wrappers \
  import wrap_openai
client = wrap_openai(
  OpenAI())
```

C · DECORATOR

## @traceable anything

Your own functions become runs; nesting is automatic.

```
@traceable
def answer(q):
  docs = retrieve(q)
  return llm(q, docs)
```

Real apps mix B and C: wrap the model client, decorate your pipeline steps. TypeScript uses **traceable()**; Java uses **Tracing.traceFunction**.

02 · CONFIGURE

# @traceable, all the knobs

```python
from langsmith import traceable
@traceable(
    name="retrieve_docs",   # label in UI
    run_type="retriever",   # how to render
    tags=["rag", "v2"],    # quick filters
    metadata={"top_k": 2}, # searchable k/v
)
def retrieve_docs(question): ...
# add context at CALL time, no code change
answer_question(q, langsmith_extra={
    "metadata": {"user_id": "u-42"},
    "tags": ["beta"], "run_id": my_uuid})
```

### Tags

Short labels: version, feature, experiment

### Metadata

Key-value context: user, tenant, model, release

### run_id

Pick it yourself to attach feedback later

02 · CONFIGURE

# Threads and context: group, route, switch off

## Threads = conversations

Add one of **session_id**, **thread_id** or **conversation_id** to metadata. Every turn with the same id appears in the Threads tab.

```
tid = str(uuid.uuid4())
for q in chat_turns:
  answer_question(q,
    langsmith_extra={
      "metadata":
      {"session_id": tid}})
```

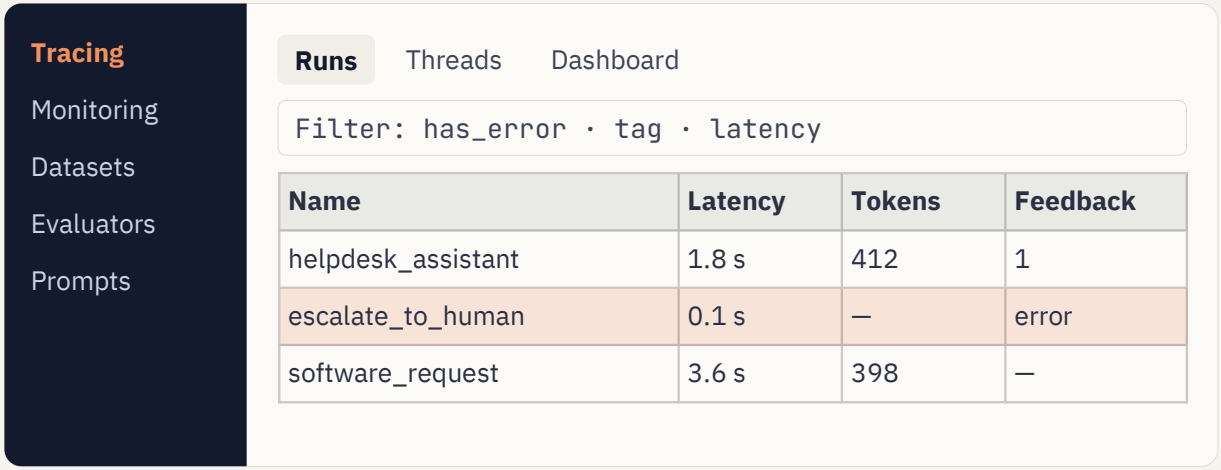
## tracing_context = scoped settings

Change project, tags or metadata for a block of code — or turn tracing off for it.

```
import langsmith as ls
with ls.tracing_context(
  project_name="helpdesk-staging",
  tags=["smoke"]):
  run_smoke_tests()
with ls.tracing_context(enabled=False):
  run_load_test()
```

03 · OBSERVE

# A tour of the tracing UI




1. **Project** — pick app + environment
2. **Runs table** — one row per trace, sortable
3. **Filter bar** — errors, tags, metadata, latency
4. **Trace view** — tree, waterfall, inputs, outputs
5. **Threads** — whole conversations
6. **Dashboard** — trends over time

Simplified sketch — labels in the live UI may differ slightly between releases.

03 · OBSERVE

# Find the needle: filter recipes

| Question | Filter (same syntax in UI and SDK) |
|---|---|
| Which requests failed? | eq(status, "error") |
| What’s slow? | gt(latency, "2s") |
| Only the beta release | has(tags, "beta") |
| One customer’s traffic | and(eq(metadata_key, "user_id"), eq(metadata_value, "u-42")) |
| Thumbs-down answers | and(eq(feedback_key, "user_thumbs"), eq(feedback_score, 0)) |
| A specific step | eq(name, "retrieve_docs") |

**Trace filter vs run filter:** filter on the root (“traces where the answer was bad”) or on any child (“traces where retrieve_docs returned 0 docs”).

**Save the view.** Saved filters become your team’s shared “bad answers” and “slow traces” queues.

03 · OBSERVE

# Tokens and cost: where the money goes

1. An **llm** run reports **input / output tokens**
2. LangSmith matches **provider + model name** to a price
3. Cost rolls up: run → trace → project → dashboard

Wrappers and LangChain do this for you. Custom or self-hosted models? Report usage yourself (right).

```python
@traceable(run_type="llm", metadata={
  "ls_provider": "openai",
  "ls_model_name": "gpt-4o-mini"})
def my_llm(messages):
  out, usage = call_model(messages)
  get_current_run_tree().set(
    usage_metadata={
      "input_tokens": usage.inp,
      "output_tokens": usage.out,
      "total_tokens": usage.total})
  return out
```

03 · OBSERVE

# From single traces to the big picture

### Prebuilt dashboard

Every project gets one: trace volume, error rate, latency percentiles, tokens, cost, feedback scores over time.

### Custom dashboards

Your own charts, grouped by metadata: cost per tenant, latency per model, thumbs-down rate per release.

### Alerts

Threshold rules on error rate, latency or feedback that notify your on-call channel before users complain.

Good dashboards depend on good metadata. Decide your metadata keys on day one — app_version, model, tenant, feature.

03 · OBSERVE

# Feedback: put a score on every answer

### End users
Thumbs up/down from your UI, sent through the API

### Experts
Annotation queues: route runs to reviewers with a rubric

### Automated
Online evaluators (code or LLM-as-judge) score live traffic

```python
# 1. choose the run id up front
run_id = uuid.uuid4()
answer_question(q, langsmith_extra=
  {"run_id": run_id})

# 2. later, from your /feedback API
client.create_feedback(
  run_id,
  key="user_thumbs",
  score=0,
  comment="wrong policy")
```

04 · DEBUG

# The debugging playbook: symptom → span → cause

| Symptom | Look here in the trace | Usual root cause |
|---|---|---|
| Wrong answer | retriever run → returned documents | Bad search, chunking or filters |
| Hallucination | llm run: is the fact in the input context? | Missing context, weak prompt rules |
| Exception | Red run → error tab + its exact inputs | Bad tool args, schema mismatch |
| Slow | Waterfall → the longest bar | Slow API, sequential calls, big context |
| Expensive | Token and cost columns per llm run | Huge prompts, wrong model, retries |
| Agent loops | Same tool run repeated in the tree | No stop condition, tool keeps failing |

**Golden rule:** check the **input** of the step that went wrong before blaming the model. Most “LLM bugs” are context bugs.

04 · DEBUG

# A repeatable debug loop




Steps 1–4 happen in LangSmith, without touching code.

**Step 6 is what most teams skip** — and why the same bug comes back next sprint.

04 · DEBUG

# Playground: replay a bad call, then fix the prompt

- **Open any llm run in Playground** — the exact messages and settings load in
- **Edit and rerun** — tighten the system prompt, change temperature
- **Compare models side by side** on the same input
- **Save as a versioned prompt** and pull it from code

```
# needs langchain-core
from langsmith import Client
prompt = Client().pull_prompt(
  "helpdesk-system:prod")
msgs = prompt.invoke(
  {"context": ctx, "question": q})
```

Prompt changes ship without a code deploy — and every trace records which version ran.

05 · EVALUATE

# Tracing shows what happened. Evaluation proves it’s good.

## Offline evaluation

**Before you ship**

- Curated dataset with expected outputs
- Compare versions, catch regressions
- Runs in your laptop or CI pipeline
- Answers: “is v2 better than v1?”

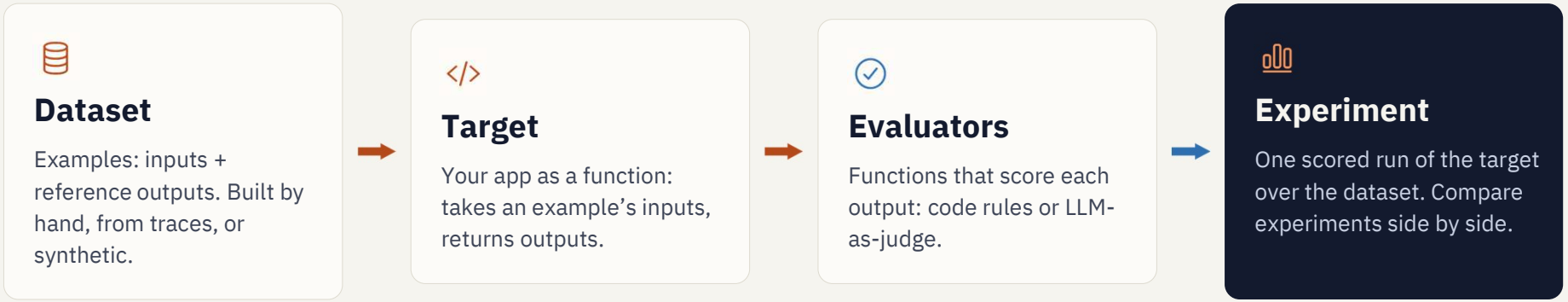
## Online evaluation

**After you ship**

- Scores real production traces
- No reference answer: reference-free checks
- Per run or per whole thread, sampled
- Answers: “is quality drifting right now?”

05 · EVALUATE

# Offline evaluation in four moving parts




Start small: **10–20 well-chosen examples** beat 1,000 random ones. Grow the set every time production shows you a new failure.

05 · EVALUATE

# Choosing an evaluator

| Type | Good for | Trade-off |
|---|---|---|
| Code / heuristic | Contains fact, valid JSON, regex, length, tool called | Fast, free, exact — but narrow |
| LLM-as-judge | Correctness, groundedness, tone, helpfulness | Flexible — costs tokens, can be wrong |
| Pairwise | “Is answer A better than B?” across two versions | Great for prompt A/B tests |
| Human | Annotation queues with a rubric for experts | Gold standard — slow and costly |
| Summary | Whole-dataset metrics: precision, recall, F1 | One number per experiment |

**Layer them:** cheap code checks on everything, LLM-as-judge for meaning, humans to calibrate the judge.

**Don’t start from zero:** LangSmith ships 30+ evaluator templates and reusable workspace evaluators (April 2026).

05 · EVALUATE

# An experiment in about 20 lines

```
client = Client()
ds = client.create_dataset("helpdesk-golden-set")
client.create_examples(dataset_id=ds.id, examples=[
  {"inputs": {"question": "When do passwords expire?"},
   "outputs": {"must_contain": "90 days"}}, ...])
# evaluator: parameters are matched by name
def correctness(outputs, reference_outputs):
  ok = reference_outputs["must_contain"] in outputs["answer"]
  return {"key": "correctness", "score": int(ok)}
def target(inputs):
  return answer_question(inputs["question"])
client.evaluate(target, data="helpdesk-golden-set",
  evaluators=[correctness],
  experiment_prefix="helpdesk-v2")
```

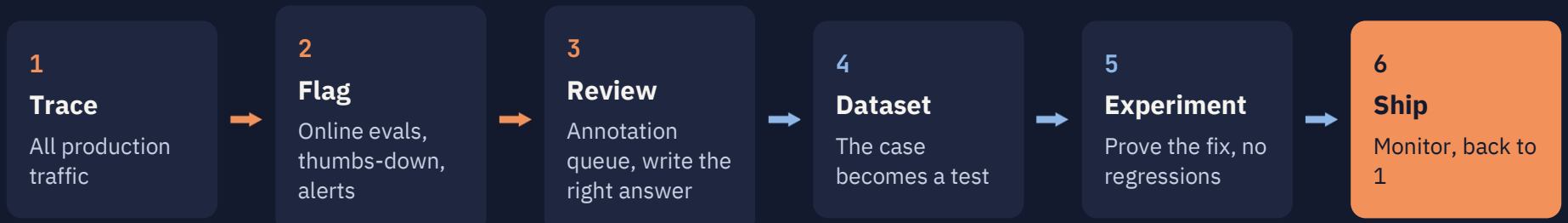
**Evaluator args** you can ask for: inputs, outputs, reference_outputs, run, example.

**Return** a bool, a number, or {key, score, comment}.

**Useful options:** max_concurrency, num_repetitions, metadata.

05 · EVALUATE

# The flywheel: production makes your tests better




Run step 5 in CI on every pull request and an AI change gets the same safety net as a code change.

06 · PRODUCTION

# Production checklist

- **One project per environment** — app-dev, app-staging, app-prod
- **Sample high volume** — LANGSMITH_TRACING_SAMPLING_RATE=0.1
- **Flush short-lived processes** — client.flush() in scripts, jobs, serverless
- **Keys in a secret manager** — separate keys per environment
- **Mask PII before it leaves** — hide_inputs / hide_outputs on the Client
- **Standard metadata keys** — app_version, model, tenant, session_id
- **Online evaluators + alerts** on the prod project from day one
- **Data residency** — cloud (US/EU), hybrid or self-hosted

06 · PRODUCTION

# Troubleshooting LangSmith itself

| You see... | Check this |
|---|---|
| No traces at all | LANGSMITH_TRACING=true? Was .env loaded before importing the app? |
| 401 / 403 errors | Wrong key, or EU account without LANGSMITH_ENDPOINT |
| Traces in “default” | LANGSMITH_PROJECT not set in that process |
| Last trace missing | Script exited too early — call client.flush() |
| Duplicate traces | Gateway callback and SDK wrapper both on for the same call |
| Flat, not nested | Inner function not decorated, or context lost in a thread pool |
| No cost shown | llm run without token usage or model name |
| Connection errors | Corporate proxy: set HTTPS_PROXY; firewall must allow the API host |

06 · PRODUCTION · INSIDE FIL

# Inside FIL: one LangSmith, three ways in

FIL runs its own LangSmith behind **FIL AI Observability**. One lsv2_ key and one host: OpenTelemetry for traces, the SDK for data, the browser for the rest.

### Traces

fil-aiobs → /api/v1/otel. AI Gateway, FilCopilot and LangGraph spans, plus your @traced steps.

### Datasets & evals

langsmith Client → /api/v1. Datasets, experiments and feedback with the same key.

### Playground

Versioned prompt commits; Evaluate mode with prebuilt LLM-as-judge evaluators.

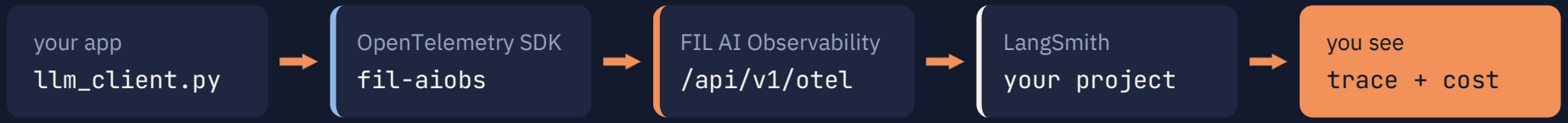
### FIL LangSmith UI

Tracing, Monitoring, Datasets & Experiments, Annotation queues, Prompts.

Ways in: `fil-aiobs` `OpenTelemetry` `langsmith SDK` `/api/v1` `Playground UI`

06 · PRODUCTION · INSIDE FIL

# How traces leave the FIL network




| | | | |
|---|---|---|---|
| 1 **SRM credentials** | 2 **LangSmith key + project** | 3 **pip install fil-aiobs** | 4 **Fill the OBS_* block** |
| 5 **00_check_setup.py** | 6 **01_check_observability.py** | 7 **Verify the trace tree** | 8 **@traced your own steps** |

LLM calls take the other path: llm_client → FIL AI Gateway with an MSAL token; a **traceparent** header keeps both paths on one trace id. Datasets and prompts use the same host's **/api/v1**.

06 · PRODUCTION · INSIDE FIL

# Steps 1–2: a token for calls, OTel for traces

## Step 1 · FilCopilot token (MSAL)

Client-credentials flow with the **SRM ticket** values. A UAT token is rejected by DEV: the scope must match the environment.

```
from msal import ConfidentialClientApplication
client = ConfidentialClientApplication(
  client_id=FIL_CLIENT_ID,     # SRM ticket
  client_credential=FIL_CLIENT_SECRET,
  authority="https://login.microsoftonline.com/"
            + FIL_TENANT_ID)
tok = client.acquire_token_for_client(
  ["https://uat.filxgptnp.fil.com/.default"])
access_token = tok["access_token"]
```

## Step 2 · Initialise OpenTelemetry (fil-aiobs)

Spans go to **FIL AI Observability**, which forwards them to your LangSmith project using your lsv2_ key.

```
from opentelemetry import trace
from fil_aiobs.telemetry import init_otel
provider = init_otel(
  endpoint=OBS_ENDPOINT,   # .../api/v1/otel
  x_api_key=LANGSMITH_API_KEY,   # lsv2_...
  project_name=LANGSMITH_PROJECT,
  console_export=False)
tracer = trace.get_tracer(__name__)
```

You don’t type either block in the lab: **llm_client.py** runs both from **.env** the first time you call get_client().

06 · PRODUCTION · INSIDE FIL

# Step 4: the .env observability block

| Variable | Purpose | Inside FIL |
|---|---|---|
| OBS_MODE | fil · otlp · off — where spans go | fil |
| OBS_ENDPOINT | OTLP traces → FIL AI Observability | …uat…/api/v1/otel |
| LANGSMITH_ENDPOINT | SDK REST API: datasets, prompts, feedback | …uat…/api/v1 |
| LANGSMITH_API_KEY | From the workspace admin dashboard | lsv2_sk_… |
| LANGSMITH_PROJECT | Project that receives the traces | rag-lab-uat-<you> |
| OBS_METADATA | Stamped on every span: group and filter | team,app,use_case |
| OBS_TRACE_LANGCHAIN | LangChain / LangGraph runs via OTel only | true |
| OBS_CAPTURE_CONTENT | Prompts and answers, or metadata only | false for client data |

**Don’t set LANGSMITH_TRACING** yourself: with OBS_TRACE_LANGCHAIN=true, llm_client turns it on in OTel-only mode.

06 · PRODUCTION · INSIDE FIL

# llm_client.py wires it up for you

```
# every lab script and the FastAPI app, unchanged
from llm_client import get_client, settings

client = get_client()   # 1st call: init_observability()
reply = client.chat.completions.create(
    model=settings.chat_model,     # azure/gpt-6-luna
    messages=[{"role": "user", "content": q}])

# -> span "chat azure/gpt-6-luna"  kind = llm
#    gen_ai.prompt / completion / usage tokens
#    + traceparent header to the FIL gateway
```

### LLM spans for free

Chat and embedding calls become llm / embedding runs with tokens and cost.

### One trace id end to end

Each /api request is the root span; traceparent links the gateway.

### Fail-safe and flushed

No collector? One warning, the app keeps running. Spans flush at exit.

06 · PRODUCTION · INSIDE FIL

# LangGraph and FilCopilot calls, same trace

## LangChain / LangGraph: no code change

With **OBS_TRACE_LANGCHAIN=true**, the LangSmith SDK runs in OTel-only mode and exports through fil-aiobs.

```
# .env
OBS_TRACE_LANGCHAIN=true      # the default

agent.invoke({"messages": [("user", q)]})

# LangSmith shows (FIL OTel route):
# POST /api/agent/chat
#  └ LangGraph
#    ├ model → azure/gpt-4o   llm · 74 tok
#    └ tools → get_weather    tool
```

## FilCopilot API: wrap the call in a span

Same MSAL token; **trace_headers()** adds traceparent so FIL-side spans join your trace.

```
from llm_client import (get_access_token, span,
                        set_output, trace_headers)
with span("filcopilot.ask", kind="llm",
          inputs={"question": q}) as sp:
    r = httpx.post(f"{FILCOPILOT_HOST}/<path>",
        headers={"Authorization":
                 f"Bearer {get_access_token()}",
                 **trace_headers()},
        json=payload)        # path: FIL docs
    set_output(sp, r.json())
```

06 · PRODUCTION · INSIDE FIL

# Step 8: trace your own pipeline steps

app/pipeline.py, or any of your own code

```python
from llm_client import traced, span, set_output, obs_context
@traced("retrieve_docs", kind="retriever")
def retrieve(question): ...
@traced("rag.run")
def answer(question):
    docs = retrieve(question)
    with span("build_prompt", kind="prompt") as sp:
        prompt = build(question, docs)
        set_output(sp, prompt)
    return llm(prompt)       # auto llm span
with obs_context(session_id=tid, user_id="u-42"):
    answer("When do passwords expire?")
```

```
$ uvicorn app.main:app --reload --port 8000
```

LOOK IN LANGSMITH

- ☐ Tree: **POST /api/chat** → rag.run → retrieve → chat
- ☐ retrieve_docs shows **documents**
- ☐ **Threads**: turns with one session_id
- ☐ Filter metadata **user_id = u-42**

**Takeaway:** @traced is the OpenTelemetry twin of @traceable.

06 · PRODUCTION · INSIDE FIL

# Steps 5–7: verify before you build

RUN

```
pip install -r requirements.txt
pip install fil-aiobs        # FIL package index
cp .env.example .env         # fill FIL_* OBS_* LANGSMITH_*
python 00_check_setup.py     # gateway + token
python 01_check_observability.py
python 01_check_observability.py --no-llm  # telemetry only
pytest tests/test_observability.py         # offline
python 02_datasets_and_evals.py --dry-run  # evals, local
python 02_datasets_and_evals.py            # → /api/v1
```

EXPECTED OUTPUT

```
OK OBS_MODE=fil
OK LANGSMITH_API_KEY is set
OK fil-aiobs SDK importable
OK Azure AD token OK
OK tracer provider started
OK chat call OK ('pong')
OK spans exported
```

Then in LangSmith: root run **obs_smoke_test** → chat, embeddings, mini_rag and a red **create_ticket**. Threads: 2 traces, one session. Any **FIX** line? Next slide.

06 · PRODUCTION · INSIDE FIL

# Reading a trace in FIL LangSmith

### Red run = error

A 401 “LiteLLM virtual key” error shows on the run, next to the input that caused it.

### Latency and tokens on every run

Badges like 4.45 s · 485 tokens show where time and money go. Fix the slowest run first.

### Tree, Threads, LLM Calls

Expand LangGraph → model → tools. Threads groups a conversation; LLM Calls lists every model call.

### Input, Output, Playground

SYSTEM / HUMAN in, AI out, tool calls included. One click replays the run in the Playground.

**Find the run → read its input → replay in the Playground → add it to a dataset.**

06 · PRODUCTION · INSIDE FIL

# Datasets in FIL: the SDK on /api/v1

02_datasets_and_evals.py (core) · FIL docs pattern

```
from llm_client import get_langsmith_client
client = get_langsmith_client()   # api_url = …/api/v1
ds = client.create_dataset("rag-lab-golden",
        description="question -> fact to retrieve")
client.create_examples(dataset_id=ds.id, examples=[
  {"inputs":  {"question": "What is the RTO of Ledgerline?"},
   "outputs": {"must_contain": "RTO of 30 minutes"}}, ...])
client.evaluate(rag_target, data="rag-lab-golden",
    evaluators=[retrieved_fact, answer_has_fact],
    experiment_prefix="rag-mqr_rrf")
```

```
$ python 02_datasets_and_evals.py      # --dry-run first
```

LOOK IN LANGSMITH

- ☐ Open dataset **rag-lab-golden**
- ☐ Every edit = a new **dataset version**
- ☐ Or **+ New Dataset** / add a trace
- ☐ Compare **rag-basic vs rag-mqr_rrf**

**Takeaway:** traces go via OTel, data via the SDK. Same key, two paths.

06 · PRODUCTION · INSIDE FIL

# No-code evaluation in the FIL Playground

| Prebuilt evaluator | Checks whether… | Use it for |
|---|---|---|
| Correctness | the answer semantically matches the reference | Golden sets with expected answers |
| Hallucination | the answer invents facts | RAG grounding |
| Answer Relevance | the output is relevant to the input | Live traffic, no reference |
| Conciseness | the answer is concise | Chat UX and token cost |
| Toxicity | the content is toxic | Any user-facing answer |
| Code Checker | the code solves the question | Code assistants |
| Plan Adherence | an agent followed its declared plan | LangGraph agents |

**How:** Playground → Evaluate mode → pick a dataset → + Evaluator → Start.

**Need more?** Custom code (Python) or a Composite score that weights several evaluators.

06 · PRODUCTION · INSIDE FIL

# Compare versions, then watch production

### Experiment compare

Correctness, P50/P99 latency, run count and error rate per version. A prompt at 100% errors stands out in one row.

### Prompt commits

Every save is a commit with a latest tag. Try commits in the Playground, roll back, or pull one from code.

### Monitoring tabs

Traces · LLM Calls · Cost & Tokens · Tools · Run Types · Feedback Scores. Group by metadata.

Group everything by your **OBS_METADATA** keys: team · app · env · use_case. That is how one project serves many dashboards.

06 · PRODUCTION · INSIDE FIL

# Troubleshooting the FIL observability route

| You see... | Check this |
|---|---|
| DISABLED – No module fil_aiobs | pip install fil-aiobs from the FIL index (outside FIL: OBS_MODE=otlp) |
| All OK, nothing in LangSmith | Project name, UAT vs PROD endpoint, key from the right workspace; wait 30 s |
| Flush timed out / SSL error | VPN on? CA_BUNDLE for the internal cert; NO_PROXY for *.ukfilcld hosts |
| Red run: 401 LiteLLM virtual key | Gateway auth, not tracing: FIL_CLIENT_* values and FIL_ENV (uat vs dev) |
| LangGraph nodes missing or duplicated | Don’t set LANGSMITH_TRACING yourself; keep OBS_TRACE_LANGCHAIN=true |
| Dataset call fails (404 / 401) | LANGSMITH_ENDPOINT must end in /api/v1, not /otel |
| llm run has no prompt text | OBS_CAPTURE_CONTENT=false: by design for sensitive data |
| No cost on llm runs | Add a price for the gateway model name in LangSmith model pricing |

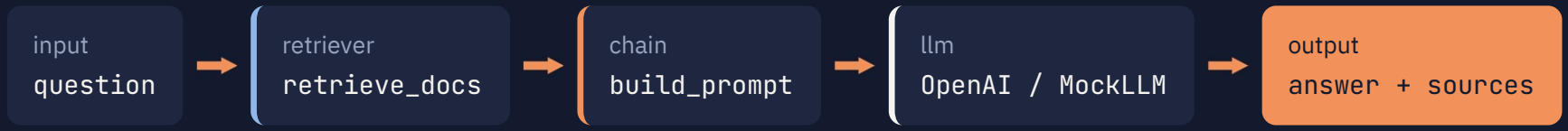
06 · PRODUCTION · INSIDE FIL

# Inside FIL: is my LLM app observable?

- **Project + OBS_METADATA** — team, app, env, use_case on every span
- **OBS_CAPTURE_CONTENT=false** wherever client or personal data flows
- **01_check_observability.py passes** before any feature work
- **Secrets live in .env** — SRM values and lsv2_ key, never in code or slides
- **pip install fil-aiobs, OBS_MODE=fil** — start on the UAT endpoint
- **OBS_TRACE_LANGCHAIN=true** — LangGraph via OTel, never direct
- **@traced on each step**; session_id via obs_context()
- **Golden dataset** — evaluate every prompt change

07 · HANDS-ON LAB · ~90 MIN

# Meet the app: Help Desk Assistant




| | | | |
|---|---|---|---|
| **0** Check setup | **1** First trace | **2** Pipeline + threads | **3** Debugging drill |
| **4** User feedback | **5** Evaluate v1 vs v2 | **6** Health report in code | **7** Production hardening |

Files: helpdesk.py (the app) + step0…step7_*.py. **version="v1"** hides a planted bug.

LAB · STEP 0 · 10 MIN

# Set up and verify the connection

RUN

```
cd langsmith-lab
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
# paste your lsv2_ key into .env
# set LANGSMITH_PROJECT=helpdesk-lab-yourname
python step0_check_setup.py
```

EXPECTED OUTPUT

```
OK LANGSMITH_TRACING
OK LANGSMITH_API_KEY
OK LANGSMITH_PROJECT
LLM mode: MockLLM
OK Connected to LangSmith
```

Any **FIX** line? Use the troubleshooting table (section 06). No OpenAI key needed — MockLLM is free.

LAB · STEP 1 · 5 MIN

# Your first trace

step1_first_trace.py

```python
from langsmith import traceable

@traceable
def greet(name: str) -> str:
    return f"Hello {name}!"

@traceable(name="shout", tags=["step1"],
    metadata={"lesson": "first-trace"})
def shout(text: str) -> str:
    return text.upper()
```

```
$ python step1_first_trace.py
```

LOOK IN LANGSMITH

- ☐ Tracing → your project → two runs: **greet**, **shout**
- ☐ Open each: Input, Output, latency
- ☐ On **shout**: the tag and the metadata
- ☐ Filter the table by tag **step1**

**Takeaway:** one decorator turns any function into an observable run.

LAB · STEP 2 · 10 MIN

# Trace the full pipeline and a conversation

**step2_trace_pipeline.py (core)**

```
thread_id = str(uuid.uuid4())
for turn, q in enumerate(conversation, 1):
    answer_question(q, version="v2",
      langsmith_extra={
        "metadata": {
          "session_id": thread_id,
          "user_id": "trainee-42",
          "turn": turn},
        "tags": ["step2", "chat"]})
```

Inside **helpdesk.py**, each step is already **@traceable** with the right run_type.

**LOOK IN LANGSMITH**

- ☐ Expand a trace: helpdesk_assistant → retrieve_docs → build_prompt → LLM
- ☐ retrieve_docs shows a **document view**
- ☐ The LLM run shows **messages, tokens, cost**
- ☐ **Threads** tab: 3 turns in one conversation
- ☐ Filter by metadata user_id = trainee-42

LAB · STEP 3 · 20 MIN

# Debugging drill: three planted issues

```
$ python step3_debug_scenarios.py  → then solve each one in the LangSmith UI only
```

ISSUE A

### Wrong answers, no errors

**Filter:** tag = issue-A

**Task:** open retrieve_docs. Which question got no documents, and why? Which got the right doc but a weak answer?

ISSUE B

### A crashing tool

**Filter:** error = true

**Task:** which run failed? Copy the exact exception and the input that caused it. Which parent called it?

ISSUE C

### A slow request

**Filter:** latency > 2 s

**Task:** in the waterfall, which child span uses most of the time? What would you do about it?

Rule for this drill: **don’t open the code**. Write each answer down — we compare on the next slide.

LAB · STEP 3 · ANSWER KEY

# What the traces reveal

### A · RETRIEVAL BUG

"delivery time for **laptops**": retrieve_docs returned **[ ]**. v1 searches only document **titles**, and the title says "laptop".

"How long does a new laptop take": right doc, but the answer skips "5 working days" → a **generation** issue.

Fix: search title + text (v2)

### B · BAD TOOL INPUT

**create_ticket** raised **KeyError: 'priority'**. Its input shows the payload has summary and docs, but no priority.

Parent **escalate_to_human** is red too: errors bubble up the tree.

Fix: pass priority, validate tool args

### C · SLOW DEPENDENCY

**check_license_server** takes ~3.0 s of the ~3.4 s trace. The LLM is not the problem.

Without the waterfall, most teams would have switched to a "faster model".

Fix: cache, timeout, or run it in parallel

Lesson: check the input of the failing step first — none of these were model bugs.

LAB · STEP 4 · 10 MIN

# Capture user feedback

**step4_user_feedback.py (core)**

```
for q, thumbs in questions.items():
    run_id = uuid.uuid4()
    answer_question(q, langsmith_extra={
        "run_id": run_id, "tags": ["step4"]})
    client.create_feedback(
        run_id, key="user_thumbs", score=thumbs,
        comment="answer was not useful")
client.flush()
```

The question “Can I work from Mars?” gets thumbs-down (score 0).

**LOOK IN LANGSMITH**

- ☐ Add the **user_thumbs** column to the runs table
- ☐ Filter: feedback user_thumbs = 0
- ☐ Open it: what did the retriever return?
- ☐ **Stretch:** add this trace to a new dataset from the UI

In a web app, return run_id with the answer; your **/feedback** endpoint calls create_feedback.

LAB · STEP 5 · 20 MIN

# Prove the fix: evaluate v1 vs v2

```
$ python step5_evaluate.py
```

Creates **helpdesk-golden-set** (10 questions + expected fact), then runs two experiments with 3 evaluators.

EXPECTED · MOCKLLM · CORRECTNESS




has_sources: v1 8/10 → v2 10/10 · concise: 10/10 both. With a real LLM, scores may vary slightly.

LOOK IN LANGSMITH

- ☐ Datasets & Experiments → helpdesk-golden-set
- ☐ Select both experiments → **Compare**
- ☐ Which examples flipped from 0 to 1?
- ☐ Click a row → see that example's full trace

**Challenge:** one v2 example still fails. Retrieval or generation? Prove it from the trace.

LAB · STEP 6 · 10 MIN

# Observability from code: a health report

step6_query_runs.py (core)

```
roots = list(client.list_runs(
    project_name=project, is_root=True,
    start_time=since))
errors = [r for r in roots if r.error]
tokens = sum(r.total_tokens or 0 for r in roots)
cost   = sum(float(r.total_cost or 0) for r in roots)
# same filter language as the UI
slow = client.list_runs(project_name=project,
    filter='gt(latency, "2s")')
```

EXAMPLE OUTPUT · YOURS WILL DIFFER

```
traces     : 41
error rate : 2%
p50 latency: 0.41s
p95 latency: 3.38s
tokens     : 3920

Failed: escalate_to_human
Slow  : software_request, ...
```

Use this pattern for daily reports, Slack digests or CI quality gates.

LAB · STEP 7 · 10 MIN

# Production hardening

```
# 1. mask PII inside your process
safe_client = Client(hide_inputs=mask,
                     hide_outputs=mask)
@traceable(client=safe_client)
def support_bot(message): ...
# 2. route a block to another project
with ls.tracing_context(
    project_name="helpdesk-lab-staging"):
    support_bot("Staging smoke test")
# 3. kill switch
with ls.tracing_context(enabled=False):
    support_bot("not traced")
```

LOOK IN LANGSMITH

- ☐ Input shows **<PHONE>** and **<EMAIL>**, never the real values
- ☐ New project **helpdesk-lab-staging** exists
- ☐ 3 calls but only **2** traces: 1 here, 1 in staging

Sampling needs no code:
LANGSMITH_TRACING_SAMPLING_RATE=0.1

LAB · COMPLETION CHECK

# You’re done when you can show…

- A nested trace with retriever, chain and llm runs
- Root cause of issues A, B and C, from traces alone
- Two experiments compared: v1 vs v2 correctness
- A trace with masked phone and email
- A thread with 3 turns in the Threads tab
- A thumbs-down trace found with a feedback filter
- A health report printed from the SDK
- A separate staging project

**STRETCH GOALS**

Add an LLM-as-judge “groundedness” evaluator · Fix the remaining v2 failure and re-run step 5 · Add a **tenant** metadata key and build a custom dashboard chart from it

08 · WRAP-UP

# Ten rules every AI developer should keep

01 Turn tracing on from the first line of code

02 One project per app and environment

03 Set run_type so the UI renders correctly

04 Agree metadata keys on day one

05 Collect feedback with a pre-chosen run_id

06 Read the input of the failing step first

07 Every bug becomes a dataset example

08 No prompt change ships without an experiment

09 Mask PII before data leaves your process

10 Watch dashboards and alerts, not just logs

Trace → Debug → Evaluate → Monitor. Repeat every release.

08 · WRAP-UP

# Keep going

### Docs

Tracing quickstart

Evaluation concepts

Cost tracking

### Next practice

Instrument your own project this week

Build a 15-example golden set for it

Add one online evaluator to production

Questions? Bring a real trace from your app — we'll debug it together.